

Vector Field - Reactive Algorithms Pt 5 - Spring with Steam

A Stream with a Spring

This one is going to have less visualizations than the first.

1. Import Libraries

```
In [11]: import numpy as np
import matplotlib.pyplot as plt
```

2. Define Constants

```
In [12]: # 2. Define constants
x_start = y_start = 0
x_end = y_end = 10
quiver_step_size = 0.2

x_source = 3
y_source = 5

robot_1_start = [8,8]
robot_2_offset = [0.3,0]
robot_3_offset = [0,0.3]

robot_step_size = 0.5
```

3. Define Flows and Robot Movements

```
In [24]: def spring_stream(x, y, x_source, y_source):

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)
```

```

    # Calculating A Source
    x_centre = x - x_source
    y_centre = y - y_source
    r = np.sqrt(x_centre**2 + y_centre**2)
    r[r==0] = 1e-6

    # Adding the uniform flow to the spring source
    u = u + x_centre / r**2
    v = v + y_centre / r**2

    return u, v

def robot_readings(x, y, x_source, y_source):

    # Centring over sink
    x_centre = x - x_source
    y_centre = y - y_source

    # Calculate the distance r from the center
    dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)

    # Adding the uniform flow to the spring source
    u = u + x_centre / dist**2
    v = v + y_centre / dist**2

    return u, v

def composite_vector(u,v):

    mag = np.sqrt(u**2 + v**2)
    angle = np.arctan2(v,u)

    return mag, angle

def plot_robot_position(pos1,pos2,pos3):

    plt.scatter(pos1[0],pos1[1], marker='o', color='blue')

```

```

plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')

def max_flow_finder(pos1, pos2, pos3, slope_y1, slope_y2, slope_x, step_size, angle):
    if np.sign(slope_y1)>0:
        pos1[1] += step_size
        pos2[1] += step_size
        pos3[1] += step_size
    else:
        pos1[1] -= step_size
        pos2[1] -= step_size
        pos3[1] -= step_size
    return pos1, pos2, pos3

def spring_finder(pos1, pos2, pos3, slope_y1, slope_y2, slope_x, step_size, angle):
    pos1[0] += step_size*np.cos(angle)
    pos1[1] += step_size*np.sin(angle)
    pos2[0] += step_size*np.cos(angle)
    pos2[1] += step_size*np.sin(angle)
    pos3[0] += step_size*np.cos(angle)
    pos3[1] += step_size*np.sin(angle)
    return pos1, pos2, pos3

def fast_flow_aligned(pos1, pos2, pos3, slope_y1, slope_y2, slope_x, step_size, angle):
    if np.sign(slope_y1)>0 and np.sign(slope_y2)>0:
        pos1[1] += step_size
        pos2[1] += step_size
        pos3[1] += step_size
    elif np.sign(slope_y1)<0 and np.sign(slope_y2)<0:
        pos1[1] -= step_size
        pos2[1] -= step_size
        pos3[1] -= step_size
    return pos1, pos2, pos3

def stagnation_finder(pos1, pos2, pos3, slope_y1, slope_y2, slope_x, step_size, angle):
    pos1[0] += step_size*np.cos(angle)
    pos1[1] += step_size*np.sin(angle)
    pos2[0] += step_size*np.cos(angle)
    pos2[1] += step_size*np.sin(angle)
    pos3[0] += step_size*np.cos(angle)
    pos3[1] += step_size*np.sin(angle)
    return pos1, pos2, pos3

return pos1, pos2, pos3

```

```

def initialize_vortex_field(x_start, x_end, quiver_step_size, y_start, y_end, x_source, y_source):
    x = np.arange(x_start, x_end, quiver_step_size)
    y = np.arange(y_start, y_end, quiver_step_size)
    X, Y = np.meshgrid(x, y)
    u, v = spring_stream(X, Y, x_source, y_source)
    return x, y, u, v

def initialize_robots(robot_1_start, robot_2_offset, robot_3_offset):
    pos1 = robot_1_start
    pos2 = [(robot_1_start[0] + robot_2_offset[0]), (robot_1_start[1] + robot_2_offset[1])]
    pos3 = [(robot_1_start[0] + robot_3_offset[0]), (robot_1_start[1] + robot_3_offset[1])]
    return pos1, pos2, pos3

```

Main Function

```

In [30]: def execute_simulation(update_robot_position, title):
    # Initialize
    x, y, u, v = initialize_vortex_field(x_start, x_end, quiver_step_size, y_start, y_end, x_source, y_source)
    pos1, pos2, pos3 = initialize_robots(robot_1_start, robot_2_offset, robot_3_offset)

    # Initialize output Plot
    plt.figure(figsize=(6, 6))
    plot_robot_position(pos1, pos2, pos3)

    # Main
    for _ in range(1, 5):
        readings = [robot_readings(pos[0], pos[1], x_source, y_source) for pos in [pos1, pos2, pos3]]
        composites = [composite_vector(u, v) for u, v in readings]

        plot_robot_position(pos1, pos2, pos3)
        pos1, pos2, pos3 = update_robot_position(pos1, pos2, pos3, slope_y1, slope_y2, slope_x, step_size, angle)

    # Plot Outcome
    plt.quiver(x, y, u, v)
    plt.title(title)
    plt.show()

```

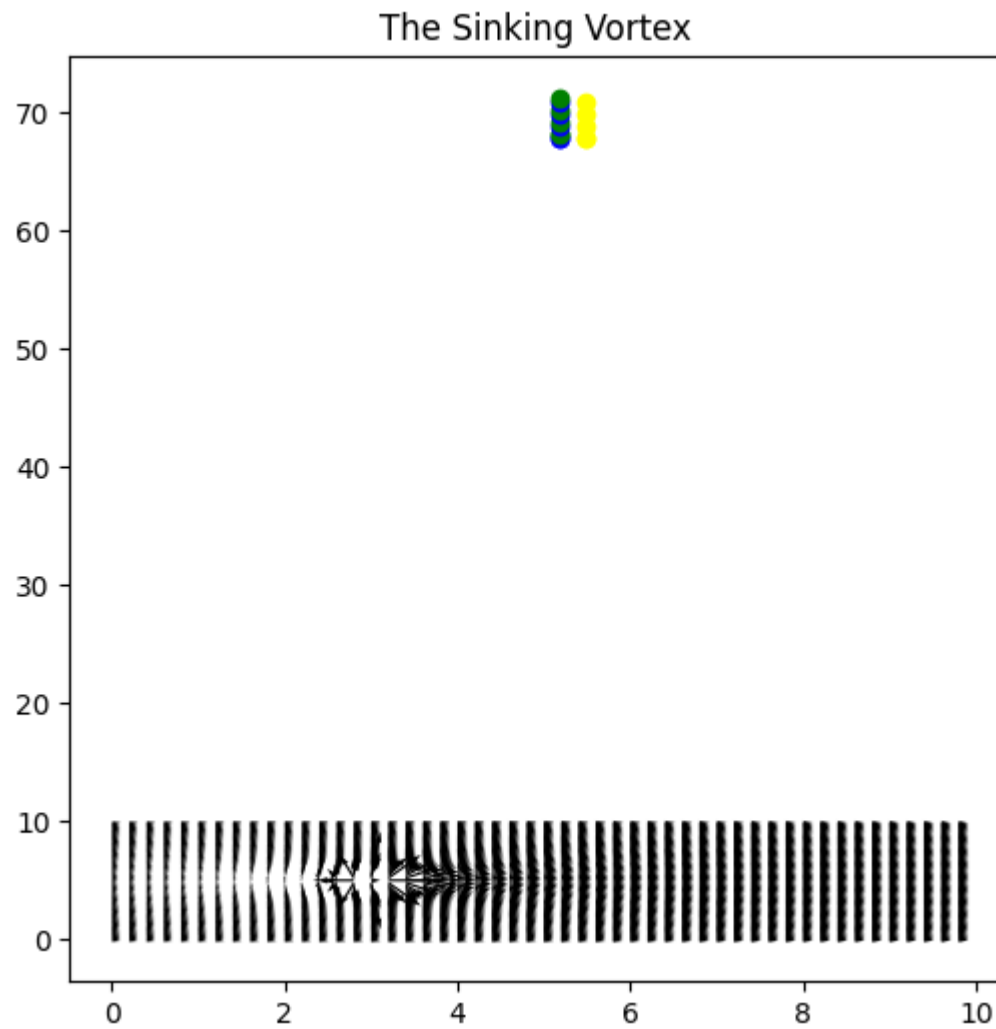
```

In [31]: # 5. Execute the simulations

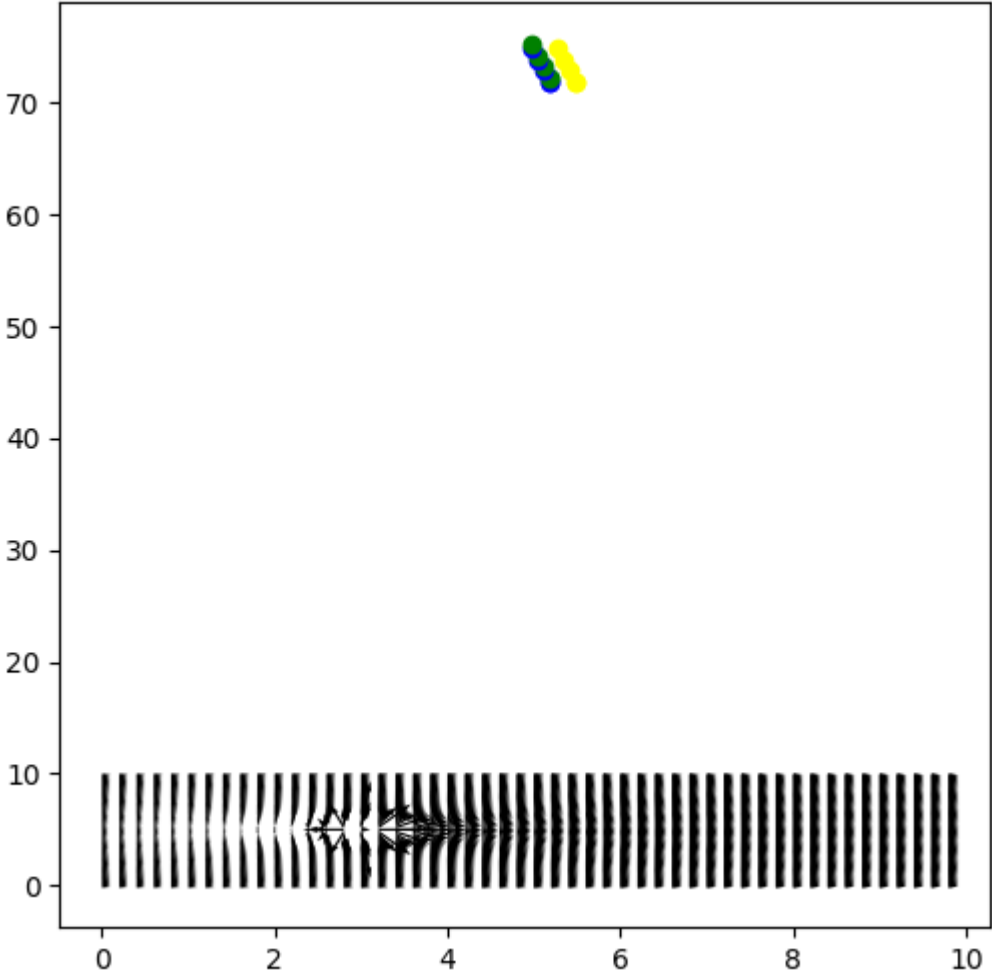
execute_simulation(max_flow_finder, 'The Sinking Vortex')
execute_simulation(spring_finder, 'The Spring in Steam')

```

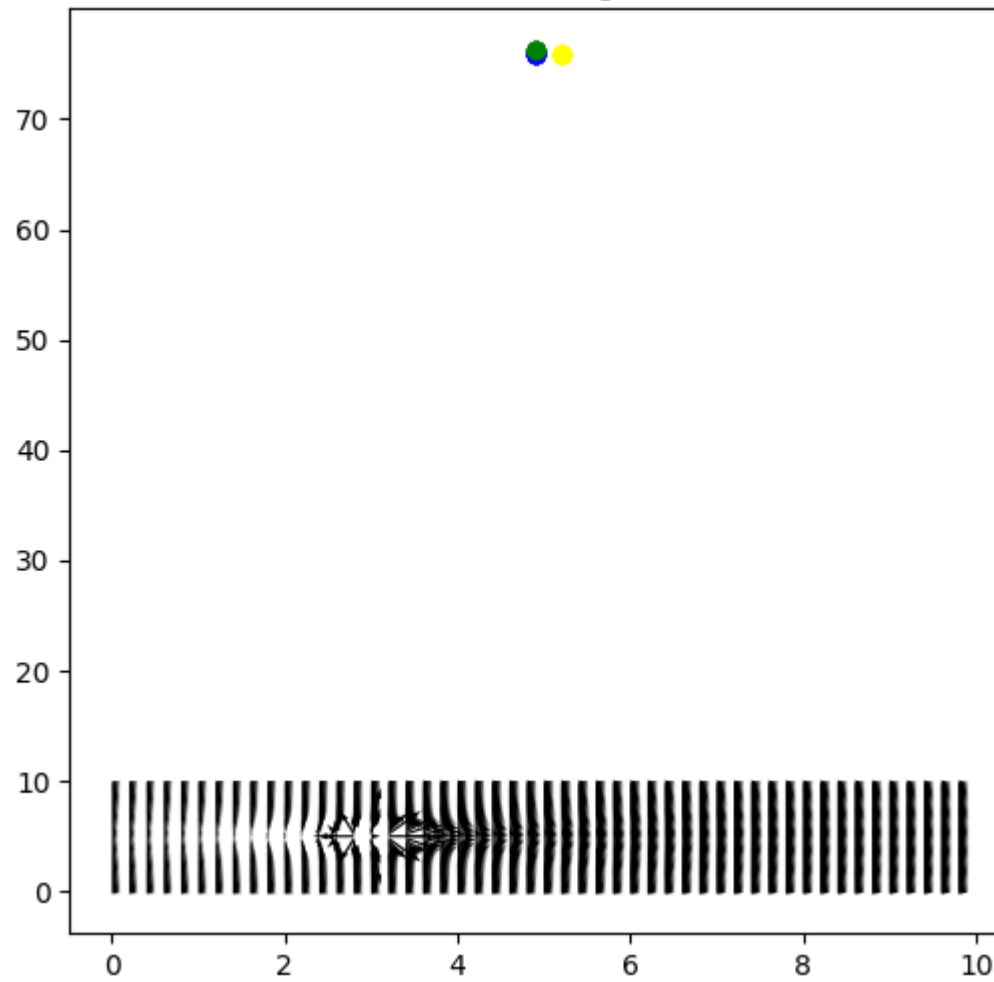
```
execute_simulation(fast_flow_aligned, 'Fast Flow Aligned')  
execute_simulation(stagnation_finder, 'Stagnation Finder')
```

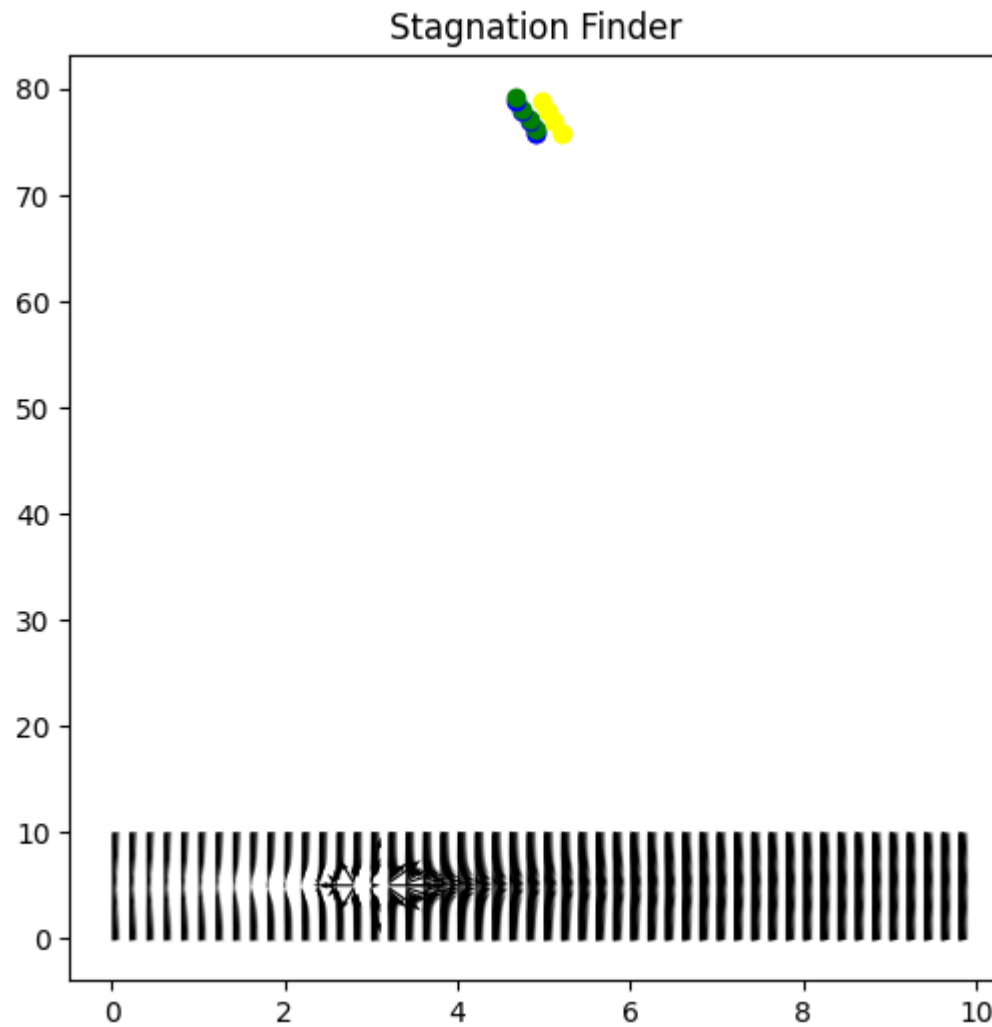


The Spring in Steam



Fast Flow Aligned





Source Finder

```
In [2]: # 4. Initialize

# Initialize The Sinking Vortex Field
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)
```



```

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)

# 5. Main

for _ in range(1,20):
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot
    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)
    [mag3, angle3] = composite_vector(u3,v3)

    # Plot Robot Position
    plot_robot_position(pos1,pos2,pos3)

    # Update Robot Position
    #angle = -(angle1 + angle2 + angle3)/3
    slope_x = (mag2 - mag1) / robot_2_offset[0]
    slope_y = (mag3 - mag1) / robot_3_offset[1]
    angle = np.arctan2(slope_y,slope_x) #+ np.pi/2

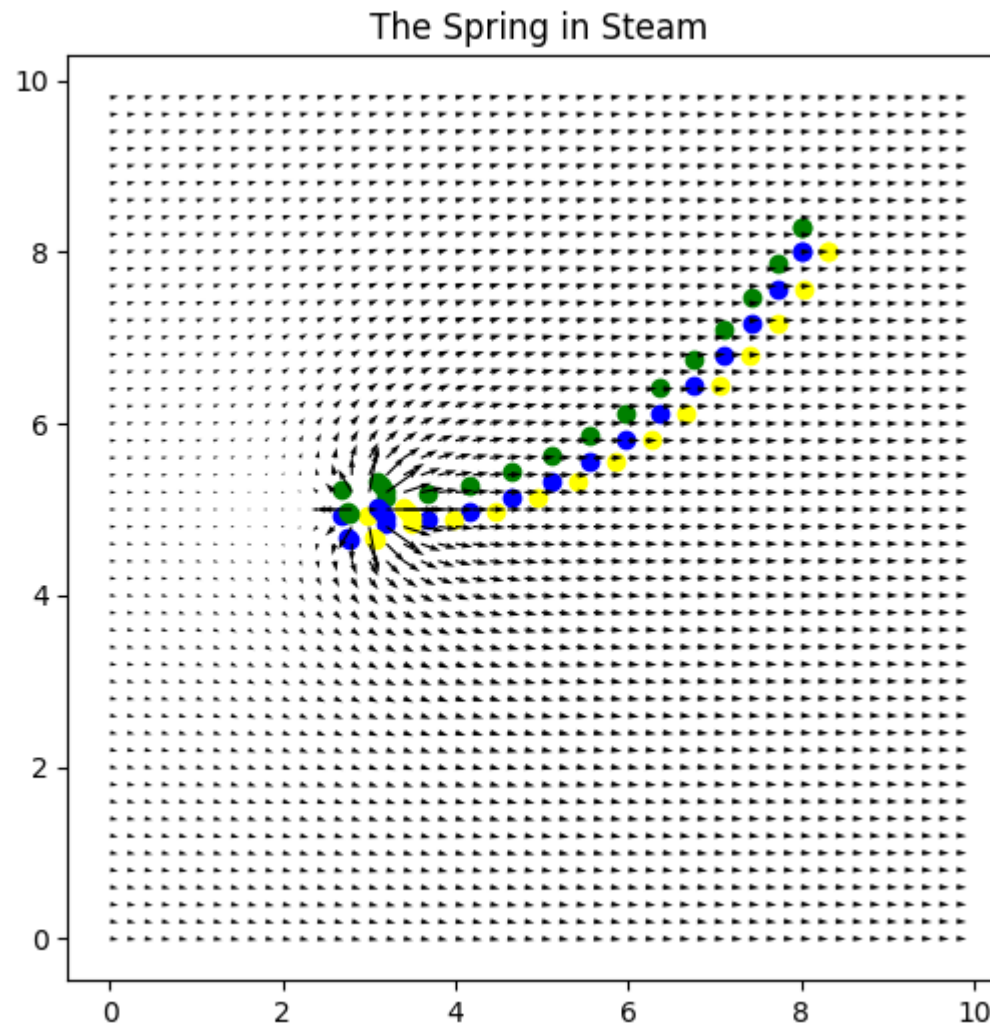
    # Update Robot Position

    [pos1, pos2, pos3] = spring_finder(pos1,pos2,pos3,angle, robot_step_size)

# 6. Plot Outcome

plt.quiver(x, y, u, v)
plt.title(' The Spring in Steam')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()

```



Fastest Flow Finder - 2 steps, straight line

```
In [10]: # 4. Initialize

# Initialize The Sinking Vortex Field
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)
```

```

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)

# Doing a rotation step prior to the main Loop.

[u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
[u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
[u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

[mag1, angle1] = composite_vector(u1,v1)
[mag2, angle2] = composite_vector(u2,v2)
[mag3, angle3] = composite_vector(u3,v3)

angle = (angle1+angle2+angle3)/3 +np.pi/2

pos1=robot_1_start
pos2=[pos1[0] + robot_offset*np.cos(angle),pos1[1] + robot_offset*np.sin(angle)]
pos3=[pos2[0] + robot_offset*np.cos(angle),pos2[1] + robot_offset*np.sin(angle)]

plot_robot_position(pos1,pos2,pos3) # Plot Robot Position

# 5. Main

for _ in range(1,100):
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot

    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)

```

```

[mag3, angle3] = composite_vector(u3,v3)

# Plot Robot Position
plot_robot_position(pos1,pos2,pos3)

# Update Robot Position
#angle = -(angle1 + angle2 + angle3)/3
#slope_x = (mag2 - mag1) / robot_2_offset[0]
slope_y1 = (mag2 - mag1) / robot_2_offset[1]
slope_y2 = (mag3 - mag2) / robot_3_offset[1] - robot_2_offset[1]

#angle = np.arctan2(slope_y,slope_x) #+ np.pi/2

# Update Robot Position

[pos1, pos2, pos3] = fast_flow_aligned(pos1,pos2,pos3,slope_y1, slope_y2, robot_step_size)

# 6. Plot Outcome

plt.quiver(x, y, u, v)
plt.title(' The Sinking Vortex')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()

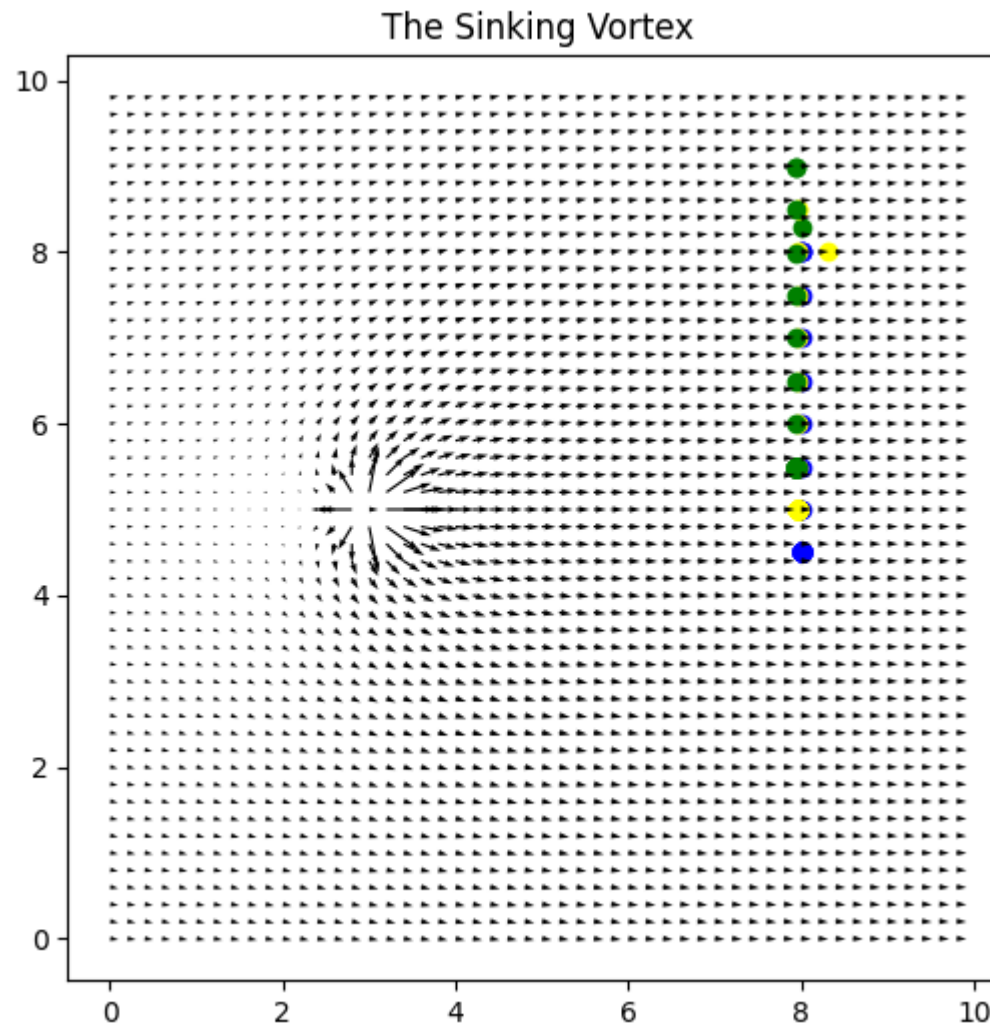
```

C:\Users\chris\AppData\Local\Temp\ipykernel_44636\1166386773.py:62: RuntimeWarning: divide by zero encountered in double_scalars

```

    slope_y1 = (mag2 - mag1) / robot_2_offset[1]

```



Lowest Magnitude Stagnation Point Finder

```
In [4]: # 4. Initialize

# Initialize The Sinking Vortex Field
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)
```

```

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)

# 5. Main

for _ in range(1,11):
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot

    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)
    [mag3, angle3] = composite_vector(u3,v3)

    # Plot Robot Position
    plot_robot_position(pos1,pos2,pos3)

    # Update Robot Position
    #angle = -(angle1 + angle2 + angle3)/3
    slope_x = (mag2 - mag1) / robot_2_offset[0]
    slope_y = (mag3 - mag1) / robot_3_offset[1]
    angle = np.arctan2(slope_y,slope_x) - np.pi/2

    [pos1, pos2, pos3] = stagnation_finder(pos1,pos2,pos3,slope_y, slope_x, robot_step_size)

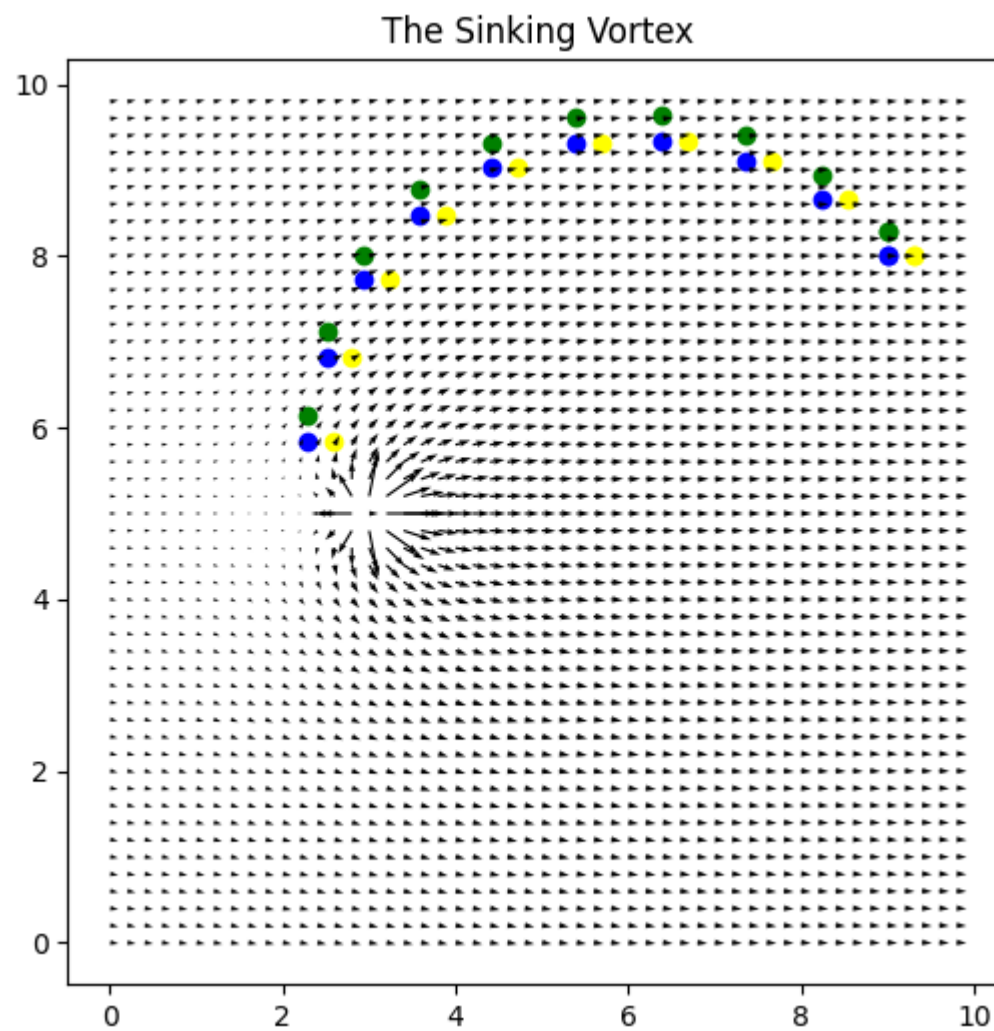
    if mag1**2 +mag2**2 + mag3**2 < 3.5 : # and slope_y**2<0.01:
        break

# 8. Getting to the stagnation point.

# 6. Plot Outcome

```

```
plt.quiver(x, y, u, v)
plt.title(' The Sinking Vortex')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()
```



In []: